

ASSESSMENT

C01- AT3

Name : B. Paneesha

Reg.No: 192472206

Course Code : CSA6502

Coure Name : Generative AI and large

Question 1: Tokenisation (5 marks). The word **"DISAGREEMENT"** is uncommon enough that a sub-word tokeniser would likely split it into smaller pieces rather than keep it whole.

(a) Split **"DISAGREEMENT"** into 3 reasonable sub-word pieces (there's no single **"correct"** answer — justify your split).

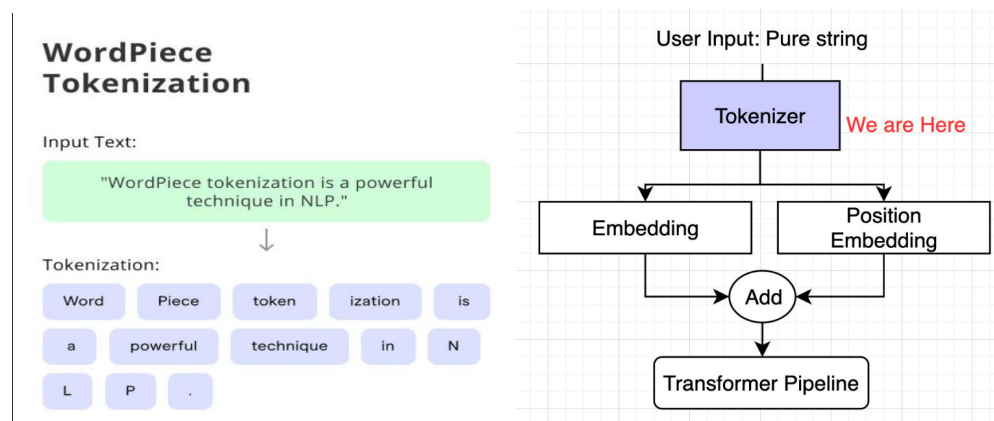
(b) In 1-2 sentences, explain WHY sub-word tokenisation (rather than whole-word tokenisation) helps a model handle a word like this one, even if the model has never seen the whole word before in training.

Question 1: Tokenisation

What is Tokenisation?

Tokenisation is the process of breaking a sentence or word into **smaller pieces called tokens** so that a transformer model (like ChatGPT) can process it.

Instead of storing every possible English word, the model stores **common sub-words**.



Example

Word:

DISAGREEMENT

It can be split into:

- **DIS**
- **AGREE**
- **MENT**

Why this split?

- **DIS** = prefix meaning "not" or "opposite"
- **AGREE** = root word
- **MENT** = suffix that changes a verb into a noun

Together:

DIS + AGREE + MENT = DISAGREEMENT

Another example:

UNHAPPINESS

↓

UN + HAPPY + NESS

Part (b)

Why do we use sub-word tokenization?

Imagine the model has **never seen the word "DISAGREEMENT"** during training.

It already knows:

- DIS
- AGREE
- MENT

So it combines these meanings and understands the new word.

Example

Suppose the model knows

- PLAY
- ER

When it sees

PLAYER

it understands

PLAY + ER

without needing PLAYER to be stored separately.

Benefits

- Smaller vocabulary
- Can understand new words
- Handles rare words easily
- Reduces memory usage

Question 2: Embeddings (5 marks). Consider the word pairs “dog” and “puppy” and “dog” and “truck”.

(a) Which pair would you expect to have HIGHER cosine similarity in embedding space, and why?

(b) In your own words, what does an embedding actually represent, and how is it different from a one-hot vector?

Question 2(a)

Which pair has higher cosine similarity?

- **Dog & Puppy**
- **Dog & Truck**

Answer:

 **Dog and Puppy**

Why?

A **puppy is a young dog**, so they have similar meanings and often appear in similar contexts (e.g., "pet," "bark," "tail").

A **truck** is a vehicle, so it is unrelated to a dog.

Therefore, the embeddings of **dog** and **puppy** are much closer together.

What is Cosine Similarity?

Cosine similarity measures how similar two embedding vectors are.

Its values range from:

Value Meaning

- 1 Exactly the same direction (very similar)

Value Meaning

0 No similarity

-1 Opposite directions

Example

Dog ↔ Puppy = 0.96  Very Similar

Dog ↔ Truck = 0.12  Not Similar

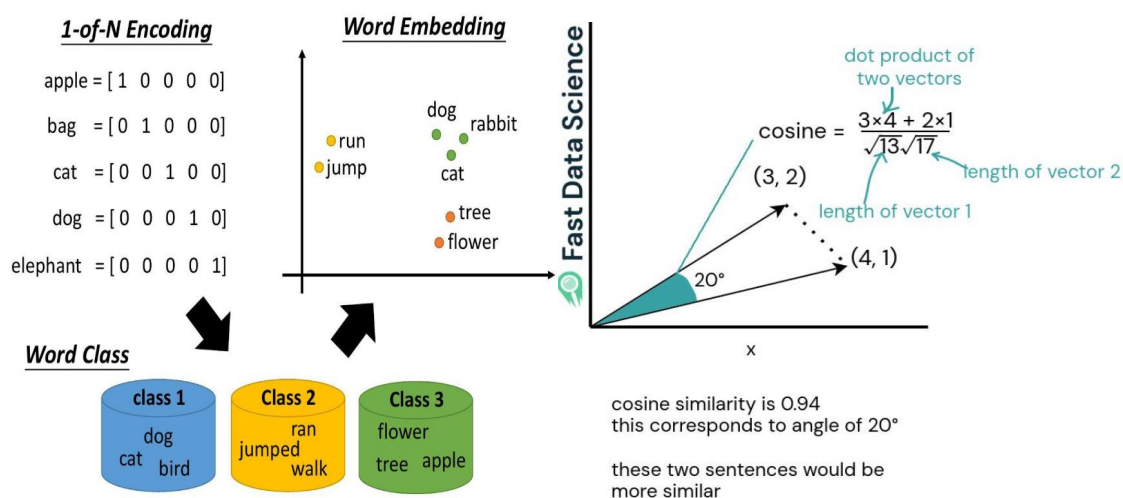
A higher cosine similarity means the words have more similar meanings.

Question 2(b)

What does an embedding represent?

An **embedding** is a dense vector of numbers that represents the **meaning and relationships** of a word based on how it is used in language.

Words with similar meanings have similar embeddings.



Example

The words **king** and **queen** have similar embeddings because both relate to royalty.

Similarly:

- Dog ↔ Puppy  Similar
- Car ↔ Truck  Similar
- Dog ↔ Truck  Not Similar

What is a One-Hot Vector?

A **one-hot vector** simply identifies a word's position in the vocabulary.

Suppose the vocabulary is:

1. Dog
2. Cat
3. Truck
4. Car

Their one-hot vectors are:

Word One-Hot Vector

Dog [1, 0, 0, 0]

Cat [0, 1, 0, 0]

Truck [0, 0, 1, 0]

Car [0, 0, 0, 1]

Here, every word is treated as completely different.

The model cannot tell that **dog** and **puppy** are related.

Embedding vs One-Hot Vector

Feature	One-Hot Vector Embedding	
Stores meaning?	No	Yes
Similar words are close?	No	Yes
Size	Very large	Smaller and dense
Learns relationships?	No	Yes

Real-Life Analogy

Imagine a classroom.

- A **one-hot vector** is like a **roll number**. It identifies a student but tells you nothing about them.

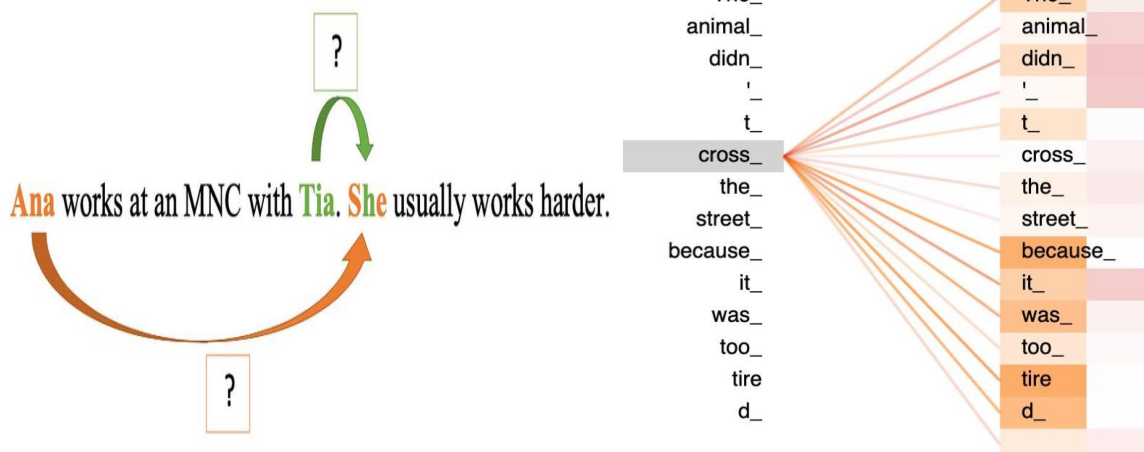
- An **embedding** is like a **student profile**. It includes information such as interests, skills, and personality, allowing you to find students with similar characteristics.

Similarly, embeddings help a Transformer understand how words are related.

Question 3: Self-Attention Worked Example (5 marks). Read the sentence: "The city council refused the protesters a permit because they feared violence.";

- (a) The word "they" should attend most strongly to which other word in the sentence? Circle one: "city council" / "protesters";
- (b) Explain in 2-3 sentences WHY self-attention (not a simple word-order rule) is needed to correctly resolve what "they" refers to here.

(a) The word "they" should attend most strongly to which word?



Answer: City council

Why?

The sentence says:

The city council refused the protesters a permit because they feared violence.

The reason for refusing the permit is that **the city council feared violence**. Therefore, the pronoun **"they"** refers to **city council**, not the protesters.

(b) Why is self-attention needed?

Self-attention allows the Transformer to compare the word **"they"** with all the other words in the sentence and determine which one best matches based on the context. Instead of simply choosing the nearest noun, it understands the meaning of the sentence and correctly identifies

that **"they"** refers to the **city council**, because the council is the group that feared violence and refused the permit.

Simple Example

Imagine the sentence as:

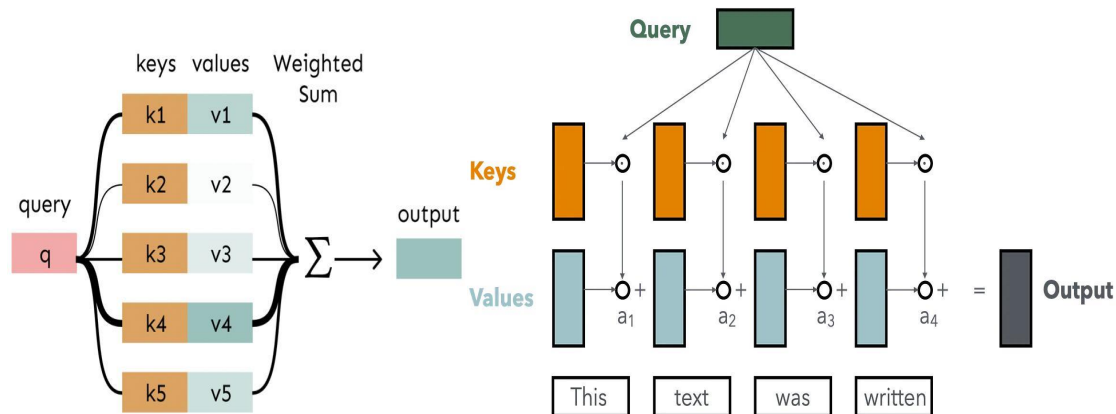
The city council → feared violence → refused permit

The protesters → wanted the permit

Since **fearing violence** explains **why the permit was refused**, **"they"** clearly refers to the **city council**.

Question 4: Query, Key, Value (5 marks). In your own words, briefly explain what the Query, Key, and Value vectors each represent in self-attention, and how they combine to produce an attention score.

Question 4: Query, Key, and Value (Q, K, V) – Explained with Images



What are Query (Q), Key (K), and Value (V)?

Self-attention in a Transformer uses **three vectors** for every word:

- **Query (Q)**
- **Key (K)**
- **Value (V)**

These three vectors help the model decide **which words are important** and **what information should be used**.

Real-Life Analogy: Searching for a Book in a Library

Imagine you go to a library looking for a book about **Artificial Intelligence**.

Step 1: Query (Q) – What am I looking for?

Your search request is the **Query**.

Query

"I need a book on Artificial Intelligence."

The Query represents **what the current word is searching for**.

Step 2: Key (K) – Book Labels

Every book has a label describing its topic.

📖 Book 1 → AI

📖 Book 2 → Mathematics

📖 Book 3 → History

These labels are the **Keys**.

The Query compares itself with every Key to find the best match.

Step 3: Value (V) – Information Inside the Book

After finding the correct book, you read its contents.

That information is the **Value**.

Book Selected

↓

Read the Contents

↓

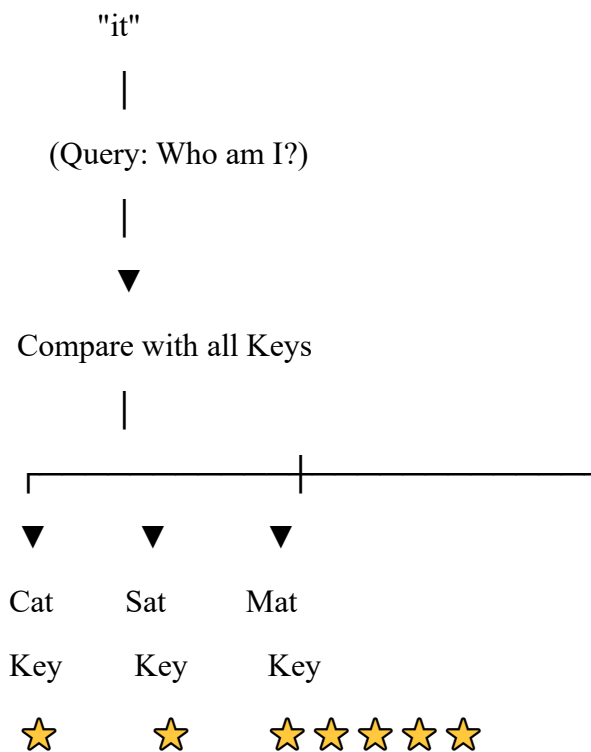
Useful Information

How Self-Attention Works

Suppose the sentence is:

The cat sat on the mat because it was soft.

The word **"it"** wants to know who it refers to.



The **Query** of **"it"** matches best with the **Key** of **"mat"**, so the model gives **mat** the highest attention.

Then, the **Value** of **"mat"** is used to understand that **"it"** refers to the **mat**.

Another Example

Sentence:

The city council refused the protesters a permit because they feared violence.

The word **"they"** asks:

Query

Who am I referring to?

It compares with the Keys:

City Council ★★★★★

Protesters ★★

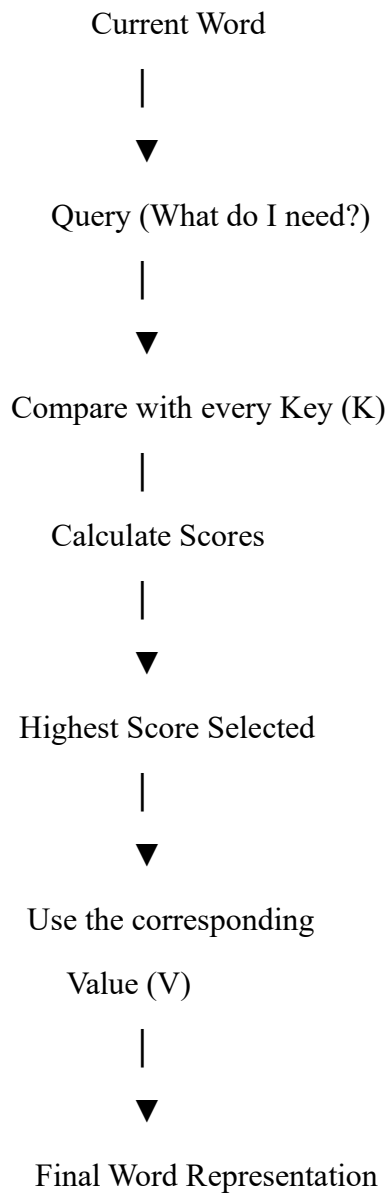
Permit ★

Highest match:

✓ City Council

The **Value** of **City Council** is then used to understand the sentence correctly.

Flow Diagram



Easy Way to Remember

Term	Easy Meaning	Example
Query (Q)	Question – What am I looking for?	"Who is they ?"
Key (K)	Label – What information does each word offer?	City Council, Protesters
Value (V)	Answer/Information – Information taken from the selected word	Information from City Council

Memory Trick

- **Q = Question**
- **K = Key to match the question**
- **V = Valuable information**